

SQL FUNDAMENTALS:

- What is a database?

Databases are an organized collection of structured information or data that is easily accessible and can be manipulated or analyzed. These data can take many forms such as user authentication data, user generated data on social media, as well as information such as watch history etc.

- Types of databases :
 - a) relational databases (aka SQL)
 - b) Non-relational databases (aka no SQL)

1) Relational Databases: Store structured data, meaning the data inserted into this database follows a structure.
 Ex: the data collected on a user consists of first-name, last-name, email, username and password. When a new user join the entry is made using this structure. This structured data is stored in rows and columns in a table.
 Relationship can be made b/w two or more tables

2) Non-relational databases: stores data in a non-tabular format. Ex: if documents are scanned which can contain varying types and quantities of data and are stored in a database that calls for a non-table format.

NOTE: Relational databases are often used when the data being stored is reliably going to be received in a consistent format, where accuracy is important such as for transactions.
 Non-relational databases are used when data being received can vary greatly in its format but needs to be collected and organised in same place such as social media platform.

BOOKS		
id	Name	Date
1	Bug Bounty	12-11-2005
2	Harry Potter	13-11-2007
3	Satanish	23-7-2013
4	Pahadi	17-1-2010

Row ←

Column ↓

Table: In which relational database stores data of a relational database

• Keys are used to relate data between two tables or I can say in relational databases relationship can be made b/w two or more tables using keys.

There are two types of keys:

- a) **Primary Key:** A primary key uniquely identifies each row in a table. Each table can only have one primary key.
- b) **Foreign keys:** They are used to connect tables. It links one table to another. It references the primary key of another table. There can be more than one foreign key column in a table.

Books				Authors	
id	Name	Date	Author-Id	id	Name
1	Harry	12.11.2005	2	1	Vickyli
2	Sadash	7.12.2006	1	2	Nikolay
3	Pahadi	3.6.2010	4	3	Loren
4	Shane	5.6.2015	3	4	Craig
5	Ishaan	7.8.2009	5	5	Daniel

Primary Key
Foreign Keys
Primary Key

In both table column 'id' is primary key because it uniquely identifies each row in a table.

Author-Id cannot be considered as primary key because same author can write two different books so in this case author-id becomes same and primary key must be unique for each row.

Here author-id is the foreign key and it can be said because every value in books-author-id must exist in authors-id.

- **DBMS (Database Management System):** Databases are controlled using DBMS which serves as an interface between the end user and the database. It is a software program that allows users to retrieve, update, manage the data being stored.
EX. of DBMS are MySQL, MongoDB, Oracle database and Maria DB.

SQL (Structured Query language) is a programming language used to query, define and manipulate the data in a relational database.

NOTE: It's a case insensitive language

- **SQL commands:**

a) How to create a database?

`CREATE DATABASE database_name;` → each command should end with ;

b) How to view all databases?

`SHOW DATABASES;` lists present databases.

c) How to use a database?

`USE database_name;` To interact with a database and run queries against it we need to set that db as active database.

d) How to delete a database?

`DROP database_name;`

e) How to create a table inside a database?

`CREATE TABLE table_name (
 example_column_1 datatype,
 example_column_2 datatype,
 example_column_3 datatype,
);`

Ex: create table book_inventory(
 book_id INT AUTO-INCREMENT PRIMARY KEY,
 book_name Varchar(255) NOT NULL,
 publication_date DATE
);

book_id is set to int as it can only be numbers and AUTO-increment mean the numbers should be in consecutive and increasing order starting from 1 and its set as the primary Key.

book name is set to variable characters (text, no, punctuation) and a limit of 255 characters and NOT NULL tells it cannot be empty.

publication_date is set as the data type date

f) How to view in which database you are currently in?
 SELECT DATABASE();

g) How to list all the tables available in a active database?
 SHOW TABLES;

h) How to know what columns are contained within a table and its datatypes?

DESCRIBE table_name or DESC table_name

i) How to alter or change the already created table?

ALTER TABLE table_name

ADD newcolumnname INT; → It is used to add a column in a table. It can also be used to rename, changing the data type in a column or removing a column.

j) How to delete a table?

DROP TABLE table-name;

k) How to create new records in a table?

INSERT INTO tablename (column^{name}~~name~~, c2^{name}~~name~~, c3^{name}~~name~~, c4^{name}~~name~~)
VALUES (c1value, c2value, c3value, c4value);

l) How to read or retrieve information from a table?

SELECT * FROM tablename;



This means that all columns should be retrieved.

If I want only specific column to be retrieved replace * with column names separated by ,.

m) How to update an existing record within a table?

UPDATE tablename

SET column~~name~~ = " ~~old~~ Its value "

WHERE ID = 10 or anything;

n) How to delete a record from the table?

DELETE FROM tablename WHERE id = anyrownumber;

- Clauses are the part of a statement that specifies the criteria of the data being manipulated. We already discussed two clauses:

FROM : used to specify the table we are accessing

WHERE : which specific which record should be used.

DISTINCT: It is used to avoid duplicate records when doing a query, returning only unique values.

Ex: SELECT DISTINCT columnname FROM tablename;

ORDER BY: It can be used to sort the records returned by a query in ascending or descending order. using function like Asc or Desc.

Example: `SELECT *`
`FROM table tablename`
`ORDER BY columnname Asc;`

HAVING: It is used to filter results created by GROUP BY using aggregate functions.

Ex: `SELECT tablename name, COUNT(*)`
`FROM Books`
`GROUP BY name`
`HAVING name LIKE '%Hack%';`

- **Operators:** They are the way to filter and manipulate data effectively.

a) **Logical Operators:** These operators test the truth of a condition and return a boolean value of true or false.

b) **LIKE Operators:** It is commonly used with WHERE in order to filter for specific patterns within a column.

Ex: `SELECT *`
`FROM tablename`
`WHERE description LIKE "%guide%";`

It returns all the columns of a ~~table~~ record in which the description column contains the word guide.

c) **AND Operator:** It used multiple conditions in a query and returns true if all of them are true.

Ex: `SELECT * FROM tablename`

`WHERE category = "offensive security" AND name = "anonymous"`

It returns the all columns of the record where the category and name matches the condition.

d) **OR operator:** The OR operator combines multiple conditions within queries and returns TRUE if atleast one of them is true.

Ex: `SELECT * FROM tablename`

`WHERE name LIKE "%Android%" OR name LIKE "%ios%";`

e) **NOT Operator:** The NOT operator reverses the value of boolean operator, allowing us to exclude a specific condition.

Ex: `SELECT * FROM tablename`

`WHERE NOT description like "%guide%";`

f) **BETWEEN Operator:** The between operator allows us to test if a value exists within a defined range.

Ex: `SELECT * FROM tablename`

`WHERE id between 2 AND 4;`

g) **NOT Equal to operator:** Ex: `SELECT * FROM tablename`
`WHERE category != "any";`

h) **Less than operator:** Ex: `SELECT * FROM tablename`

`WHERE published date < or > " "`

Greater than :

Less than equal to :

Greater than equal to :

- Functions: They can help us streamline queries and operations and manipulate data.

a) String functions: They can perform ~~operatio~~ operations on a string, returning a value associated with it.

b) CONCAT() Function: It is used to add two or more strings ~~from~~ together. It is useful to combine text from different columns.

Ex: `SELECT CONCAT(name, "is a type of", category, "book")
AS book_info FROM books;`

The original table remains unchanged it will only display temporary concatenated output.

c) Group_concat() function: It help us to concatenate data from multiple rows into one field.

Ex: `SELECT CATEGORY, GROUP_CONCAT(name separator ",") AS
books FROM books
GROUP BY category;`

It group the books by category and concatenates the titles of books within each category into a simple string.

d) Substring() function: This function will retrieve a substring from a string within a query, starting at a determined position, the length of string can also be specified.

Ex: `SELECT SUBSTRING(published-date, 1, 4) AS PUBLISHED_YEAR FROM
books;`

↓ ↘
starting extraction position length to extract

Web servers: Apache, nginx etc
DBMS: MongoDB, MySQL etc

Page No.	
Date:	

c) length() function: It returns no. of characters in a string. This include spaces and punctuations.

Ex: `SELECT length(name) AS name_length from books;`

f) COUNT() function: It returns the no. of records within an expression.

Ex: `SELECT count(*) AS total_books from books;`

g) Sum() function: It sums all values of a determined column.

Ex: `SELECT sum(price) AS total_price from books;`

h) Max() function: This function calculates the maximum value within a provided column in an expression.

Ex: `SELECT Max(published_date) as latest_book from books;`

Min() function: Returns minimum value from a column.

Ex: `SELECT Min(published_date) as earliest_book from books;`